Disjoint sets → used in dynamic graphs
↳ used to find parent
↳ Union by rank.

→ Time complexity → $O(4\alpha)$
↳ $O(1)$

→ (1,2) ✓
(2,3) ✓
(4,5) ✓
(6,7) ✓
(5,6) ✓
(3,7)

rank →

1	2	3	4	5	6	7
~~0~~ 1	0	0	~~0~~ ~~1~~ 2	0	~~0~~ 1	0

parent →

1	2	3	4	5	6	7
~~1~~ 4	~~2~~ 1	~~3~~ 1	4	~~5~~ 4	~~6~~ 4	~~7~~ 6

initially every one is a parent of themselves.

Union (u,v)

i) find ultimate parent of u & v, pu & pv.

(ii) find rank of ultimate parents

(iii) now connect smaller rank to larger rank always.

initial config

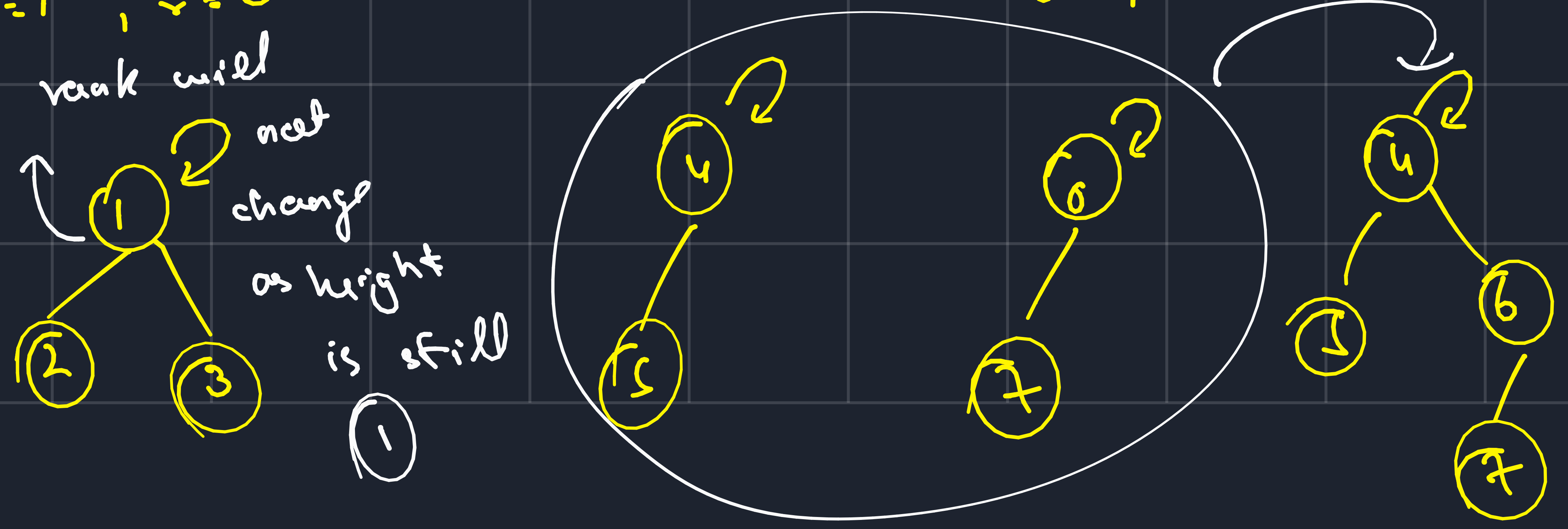
(1,2) pu = 1, pv = 1
r = 0, r = 0

(2,3) pu = 1, pv = 3
r = 1, r = 0

(4,5) pu = 4, pv = 5
r = 0, r = 0

pu = 6, pv = 7
r = 0, r = 0

(5,6)
pv = 4, pv = 6
r = 1, r = 1



rank will not change as height is still 1

→ finding parent as of now will take log(n) time.

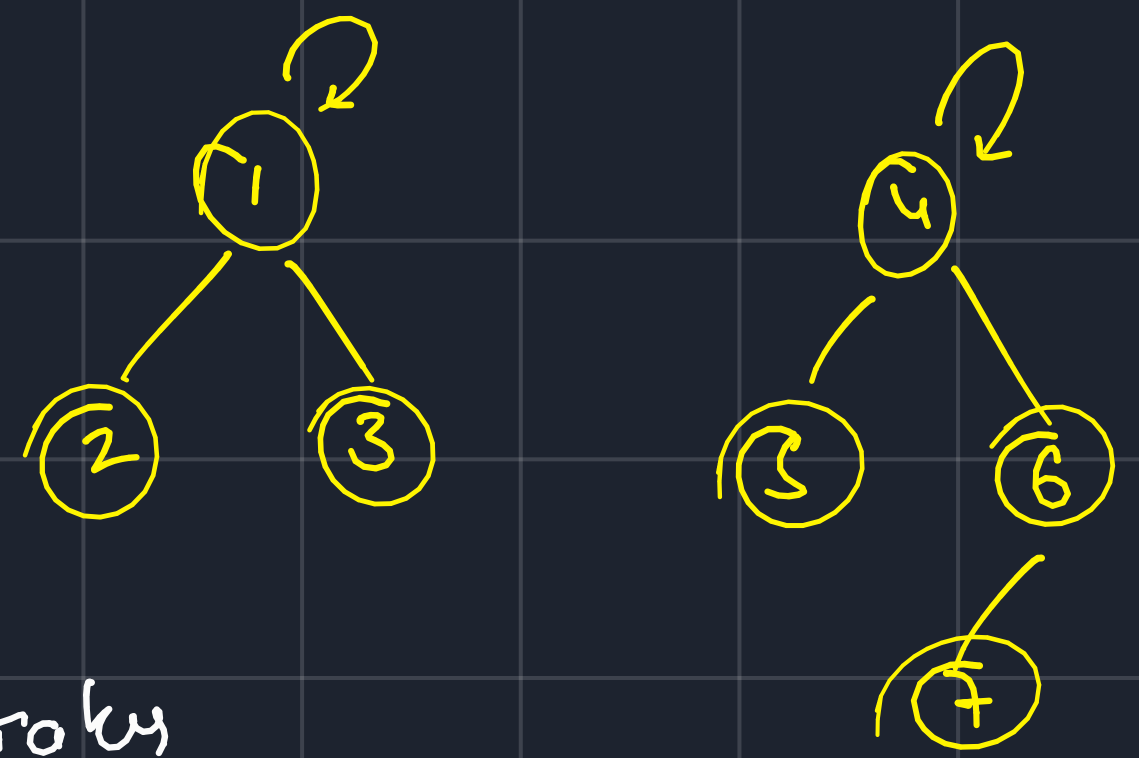
↳ Path compression is applied to make it constant.

pu = 7

7 → 6 → 4

→ currently

↳ it takes log(n) space



(3,7)

pu = 1   pv = 4

r = 1   r = 2

→ Does 1 & 7 belong to the same component.

↳ check the ultimate parents of both

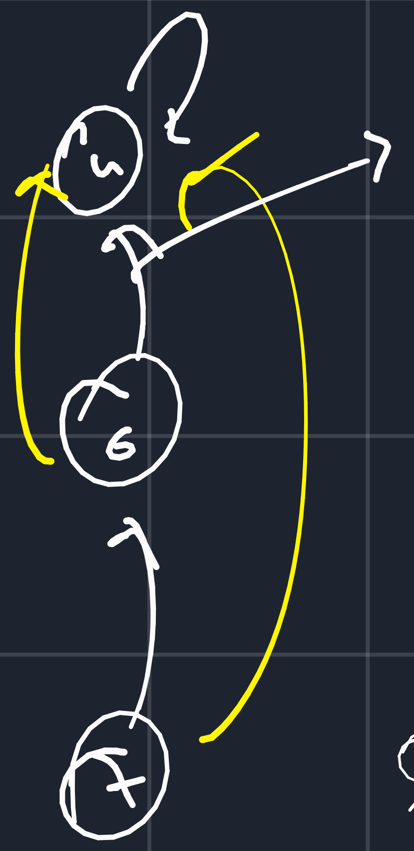
↳ same → yes

↳ ≠ → no

↳ Path compression.

↳ this skews the tree up.

↳ Ideally, the rank should be reduced



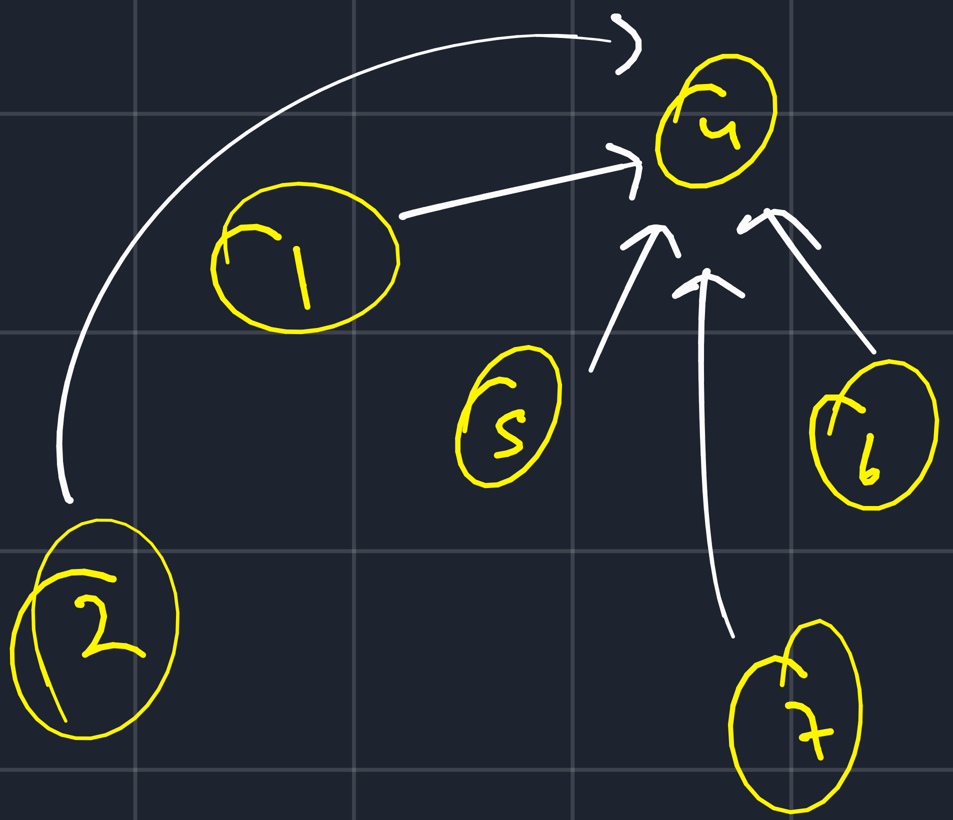
→ this link is severed

↳ all nodes store their ultimate parent.

u → 3 → 2 → 1

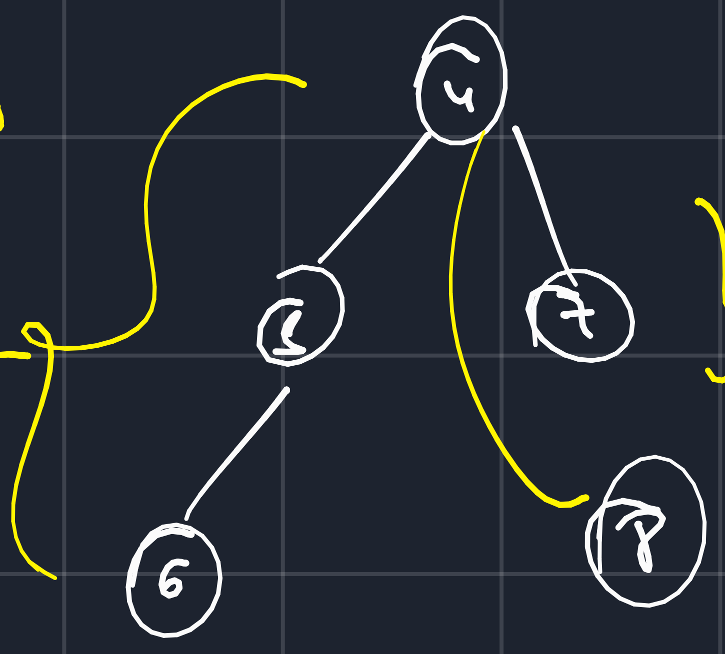
p[u] = 1 } constant

p[2] = 1



↳ Time complexity of union → O(4α)

↳ why →



p(8)

→ acc. to this we should decrease the rank

but we cannot

this is still b/w this

↳ How to path compress.

```
fn P(u)
{
    if (u == par(u))
        ret u;

    return par(u) = find.P(par(u))
}
```

f(5)

f(4)

f(3)

f(2)

f(1)